FASTMATH · REVIEW NOTES

FastMath Change Report

This report covers all the work on the review branch: bug fixes, the CMake cleanup, the API refactor, the new data layout for vectors and matrices, the quaternion clean-up, and the `LS_*` → `FM_*` rename. Read it before reviewing the diff.

Branch	`claude/fixes-and-api-refactor` (not merged)
Based on	`dev` at `d870f01` “Fix failing Quaternion test.”
Date	29 October 2026 (second revision)
Status	Ready for review. Not merged into `dev` or `main`.

148 tests passing (was 123)	**10** bugs fixed, plus 3 build issues	**0** runtime regressions on GCC; 1 on MSVC (details)	**+5%** compile time, optimized builds (details)

How to review

1. Read the behaviour changes first. They are the only changes that can break code that uses FastMath.
2. Review `VectorBase.hpp` and `MatrixBase.hpp` (the new data layout, both short), then `Vector.hpp`, `Matrix.hpp` and `Quaternion.hpp`. The same pattern is used in all three.
3. Build and test. The CMake options are now called `FM_*`:

```
git fetch origin
git switch claude/fixes-and-api-refactor      # or: git reset --hard origin/cla
cmake -S . -B build -DFM_BUILD_TESTING=ON -DFM_BUILD_PERFORMANCE=ON
cmake --build build --config Release
ctest --test-dir build -C Release
```

The branch was force-pushed. If you already pulled the first version, use the `git reset` line instead of `git switch`. The branch still has two commits: the code, and this report.

The diff against `dev` is about 1,840 lines added and 1,640 removed in 25 files. Many of the removed lines are documentation comments that moved with their functions.

Data layout: vectors and matrices

The anonymous-struct unions are gone from `Vector` and `Matrix`. Code that reads named members (`v.x`, `m[1].y`) now works in constant expressions. The compiler-specific `#pragma` warning suppressions for those headers are gone too.

Vectors

`Vector<T, 2/3/4>` stores plain members `T x, y, z, w`. Other sizes use `T elements[N]`. `VectorBase` defines `operator[]`, and `Vector` exposes it with `using base::operator[]`:

```
constexpr T& operator[]( std::size_t i ) noexcept
{
    assert( i < 3 );
    if ( __builtin_is_constant_evaluated() )
    {
        switch ( i )   // compile time: select the member
        {
        default:
        case 0: return x;
        case 1: return y;
        case 2: return z;
        }
    }

    return ( &x )[i];  // runtime: one load
}
```

This differs from the plan I described earlier (a `switch` at runtime too), because the measurements didn't support that plan:

- **A `switch` at runtime is too slow on MSVC.** GCC optimizes it away, but MSVC doesn't: matrix multiply went from 9 ns to 97 ns, and inverse from 130 ns to 836 ns. At runtime the members are therefore indexed directly. Strictly, the C++ standard doesn't define pointer arithmetic across members, but MSVC, GCC and Clang all compile it to a single load, and Unreal's `FVector` relies on the same thing. It is a much narrower dependency than the union (reading an inactive union member) was. A `static_assert` in `data()` checks that the members are tightly packed.

- **The compiler built-in is used instead of `std::is_constant_evaluated()`.** `operator[]` is inlined into every loop, and the extra wrapper function cost about 5% compile time. All three standard libraries implement `std::is_constant_evaluated()` with `__builtin_is_constant_evaluated()`, so calling it directly is portable to MSVC, GCC and Clang. I also measured a `?:` chain and a pointer-to-member table; both were slower to compile than the `switch`.
- **The binary operators no longer copy an operand.** `a + b` used to copy `a` and apply `+=`. MSVC copies the four separate members with partial loads, which cost 17% on `Vector4f` addition. The operators now build the result directly; the generated code is a single `vaddps` again.
- **`Vector4f` stays 16-byte aligned** (as before), and the SSE `dot` loads it with `_mm_load_ps( &v.x )`. The implicit conversion to and from `__m128` was removed.

Matrices

`Matrix<T, N, M>` stores `Vector<T, M> row[N]`. `m[i]` is a row, `m[i][j]` is an element, and `data()` still returns a pointer to `N × M` contiguous elements (checked with a `static_assert`). The flat `m[]` array and the named rows `X, Y, Z, W` are gone.

- **Matrix × vector** computes one `dot` per row, so `Matrix4f` uses the SSE `dot`. It's 19–21% faster on both compilers.
- **Vector × matrix** sums the rows, scaled by the vector's components. **Element-wise operators** (`+ - * /` by scalar, unary `-`) work on whole rows.
- **Matrix × matrix** stays an element-wise loop. A row-combination version was 2× slower on MSVC.
- Code inside `Matrix.hpp` reads `m.row[i]` directly instead of calling `Matrix::operator[]`, so Debug builds check each element access once instead of twice.
- The natvis visualizer shows matrices as a list of rows, for any size.

Runtime performance

The benchmarks in the new `perf/MatrixPerf.cpp` compare FastMath with glm 1.0.1. The table shows medians of 5 repetitions, before and after this round. The first version of the branch is the baseline.

BENCHMARK	MSVC BEFORE	MSVC AFTER	GCC BEFORE	GCC AFTER	GLM (MSVC / GCC)
Vector4 dot	0.62	0.63	0.20	0.21	0.62 / 0.21
Vector4 add	1.23	1.25	0.21	0.21	1.23 / 0.21

BENCHMARK	MSVC BEFORE	MSVC AFTER	GCC BEFORE	GCC AFTER	GLM (MSVC / GCC)
Vector3 cross	1.66	1.65	0.63	0.63	1.64 / 0.62
Vector3 normalize	2.45	2.47	2.52	2.53	1.42 / 1.29
Matrix4 × Matrix4	9.22	9.33	2.47	2.35	8.29 / 2.47
Matrix4 × Vector4	2.33	1.88	2.49	1.97	2.33 / 0.67
Matrix4 transpose	1.92	1.92*	0.83	0.83	1.95 / 0.82
Matrix4 inverse	130	141	110	68	12.6 / 13.0
Quaternion × Quaternion	2.26	2.28	1.08	1.08	2.31 / 1.55
Quaternion × Vector3	2.34	2.37	1.97	1.96	2.35 / 2.05

Times in nanoseconds. MSVC: VS 18, Release, AVX2. GCC: 11.4 in WSL, Release, AVX2. * Transpose alternates between 1.9 and 2.7 ns from run to run with the same binary on MSVC, so it's shown at its usual value.

- **The only regression is `inverse` on MSVC (+8%).** On GCC it is 38% faster. Both compilers are about 10× slower than glm because the general cofactor algorithm builds a smaller matrix for every minor. Closed-form 3×3 and 4×4 inverses are the fix; that's listed for the next round.
- `normalize` is about 2× slower than glm on both compilers, unchanged by this round. Also listed for the next round.
- On GCC, glm's matrix × vector is still 3× faster than FastMath's.

Compile time

The probe compiles one file that includes every header and uses each type and function (MSVC, 5 runs per measurement, two rounds, medians):

BUILD	UNIONS (BEFORE)	PLAIN MEMBERS (AFTER)	CHANGE
Debug (`/Od`)	373 ms	386 ms	+3%
Release (`/O2 /DNDEBUG`)	520 ms	547 ms	+5%
`/O2` with asserts enabled	523 ms	595 ms	+14%

Parsing costs the same. The difference is optimizer time: the generic loops used to read an array directly and now call `operator[]`, which must be inlined. The last row is an unusual configuration (CMake's Release and RelWithDebInfo define `NDEBUG`), but it shows that the `assert` in every `operator[]` costs something. The plain-members layout can't get back to the union's compile times; I got the Release cost from +12% down to +5%.

Quaternion clean-up

- **Unary minus is documented:** `-q` is the *same* rotation as `q`. For the opposite rotation, use `conjugate` or `inverse`.
- **New:** **`sameRotation( a, b, epsilon = 1e-6 )`**. It returns `true` when two unit quaternions represent the same rotation, including `q` and `-q`. It checks `|dot(a, b)| ≥ 1 − epsilon`; the default tolerance is about 0.16°.
- Quaternions still use the anonymous-struct union in `w, x, y, z` order. You didn't ask for that to change in this round; see the open decisions.

Rename: `LS_*` → `FM_*`

- **Preprocessor:**
  - `LS_HANDEDNESS`, `LS_LEFT_HANDED`, `LS_RIGHT_HANDED`
  - `LS_DEPTH_RANGE`, `LS_ZERO_TO_ONE`, `LS_NEGATIVE_ONE_TO_ONE`
  - `LS_COMPILER` and `LS_COMPILER_*`
  - `LS_SSE` through `LS_SSE4`, `LS_AVX`, `LS_AVX2`
  - `LS_DISABLE_INTRINSICS`

  All are now `FM_*`, with the same suffixes.
- **CMake:**
  - Options: `FM_BUILD_EXAMPLES`, `FM_BUILD_TESTING`, `FM_BUILD_PERFORMANCE`, `FM_ENABLE_AVX2`, `FM_ENABLE_FAST_MATH`.
  - Variables: `FM_VERSION*`, `FM_WARNINGS`, `FM_INC`.
  - The CI workflow now passes `-DFM_BUILD_TESTING=ON`.
- The target names stay `FastMath` and `FastMath::FastMath`, and the namespace stays `FastMath`.

Bug fixes

FIXED wrong result before BUILD compile or warning problem

WHERE	PROBLEM	FIX	TEST
FIXED `Matrix * Matrix`	The loops used `j < M` and `k < P` (swapped). Square matrices worked by accident; non-square products were wrong or read out of bounds.	Loop `j` over `P` and `k` over `M`.	`NonSquare_Multiplication`, `…2`
FIXED `dmat2x4`	Declared as `Matrix<double, 2, 3>`.	Now `Matrix<double, 2, 4>`.	Instantiation
FIXED `swizzle(v, x, y, z, w)`	The 4-component overload returned `Vector<T, 3>`, dropping `w`.	Returns `Vector<T, 4>`.	`Swizzle4`
FIXED Quaternion unary `-`	Returned the conjugate (`w, -x, -y, -z`) instead of the negation.	Negates all four components.	`UnaryMinus`
FIXED `slerp`	Did the near-parallel check before flipping `q1` to the shortest path, so it could take the long way round. The lerp fallback wasn't normalized. The bug above also made the flip produce a conjugate.	Flip first, then check, and normalize the lerp fallback.	`Slerp_ShortestPath`, `Slerp_NegativeDot`
FIXED `pitch(q)`	Used `q.x * q.z` where the formula needs `q.z * q.z`. It was only correct when yaw and roll were zero, which is the only case the old test covered. Euler (0.3, 0.2, 0.1) gave a pitch of 0.2989 instead of 0.3.	Use `q.z * q.z`, matching glm.	`Pitch_Combined`
FIXED Integer matrix division	`/` and `/=` multiplied by `T(1) / s`, so `Matrix2i / 2` was a zero matrix.	Divide each element, the same way `Vector` does.	`Integer_Division`
FIXED Matrix divide by zero	`M / 0` returned a zero matrix, while `M /= 0` silently did nothing.	Both now `assert( s != 0 )`, the same as `Vector` and `Quaternion`.	n/a (asserts)

WHERE	PROBLEM	FIX	TEST
FIXED `Transform`	The constructor set `isDirty = false` while the matrix still held the identity, so `getMatrix()` ignored the constructor's arguments.	`isDirty { true }`.	4 new `Transform` tests
FIXED `deadzone`	Only defined in the duplicate header `Math.hpp`, which wasn't included anywhere.	Moved to `Common.hpp`, which now includes `<cmath>`; `Math.hpp` deleted (its other contents were duplicates).	`Deadzone`
BUILD `Matrix.hpp`	Missing `<initializer_list>` and `<utility>`; `inverse` used an `int` loop index (sign-compare warnings).	Added the includes; `std::size_t` loop.	GCC `-Werror`
BUILD Asserts	`assert( ( "msg", cond ) )` failed GCC Debug with `-Werror=unused-value`.	`assert( cond && "msg" )`.	GCC Debug
BUILD Old names	`ls::` and `LightSpeed` in comments and benchmarks, and `LS_*` macros.	Renamed to `FastMath` / `FM_*`.	n/a

Build system (CMake)

- **FastMath is a header-only `INTERFACE` library** with the alias `FastMath::FastMath`. `src/` (an empty shared library) was removed. The C++20 requirement and include path come with the target: `target_link_libraries( MyGame PRIVATE FastMath::FastMath )` is all a user needs.
- **Options**
  - `FM_BUILD_EXAMPLES` (default ON only when FastMath is the top-level project)
  - `FM_BUILD_TESTING` (OFF)
  - `FM_BUILD_PERFORMANCE` (OFF)
  - `FM_ENABLE_AVX2` (default ON only at top level): passes `/arch:AVX2` or `-mavx2` to every target that links FastMath.
  - `FM_ENABLE_FAST_MATH` (OFF): `/fp:fast` or `-ffast-math`.

  The unused documentation option and the global `BUILD_SHARED_LIBS=ON` were removed.

- **Safe to embed with FetchContent or `add_subdirectory`.** Output directories, `/MP` and solution folders are set only when FastMath is the top-level project. Output now goes to the build tree, not to `bin/` and `lib/` in the source tree.
- **Warnings** (`/W4 /WX` or `-Wall -Wextra -Werror -pedantic`) apply only to FastMath's own tests, examples and benchmarks, never to user targets.
- **MSVC:** `FastMath.natvis` is attached to every target that links FastMath.
- **Tests:** they link `GTest::gtest_main` and set `gtest_force_shared_crt` (this fixes the MSVC runtime-library mismatch). They gained `Instantiation.cpp` (explicit instantiation of every class template, formerly `src/FastMath.cpp`) and `TransformTests.cpp`.
- **Benchmarks** fetch glm 1.0.1 and google benchmark v1.8.3 with FetchContent. `MatrixPerf.cpp` (previously empty) now has matrix, quaternion and vector benchmarks, each with a glm counterpart.

API refactor

Binary operators are hidden friends

In `Vector`, `Matrix` and `Quaternion`, every binary operator (`+ - * / ==`) is a `friend` defined inside the class. Compound assignment and unary `+`/`-` stay members.

- These operators are found only through argument-dependent lookup. Overload sets stay small, and scalars work on both sides of every type (`2.0f * v`, `2.0f * M`, `2.0f * q`).
- Removed: the namespace-scope `operator*` templates; `Quaternion::operator!=` (C++20 generates it from `==`); `Matrix`'s `operator<=> = default`, which the union base had already turned into a deleted function.

Scalar parameters are `T`

The operators take `T s` instead of `template<ConvertibleTo<T> U>`, so there is one instantiation per type. The constructors keep their templates, so `Vector3f{ i, j, k }` with `int` variables still compiles without narrowing errors.

The helper structs are gone

`Vector_Cross/Dot/Length/Normalize/Functor1/Functor2` and `Matrix_Transpose/Determinant/Inverse` are plain function templates that use `if`

`constexpr`. `adjugate` is now public. The element-wise trig functions are direct loops; they no longer take the address of `std::cos` and similar functions.

Compile-time support

- `ZERO`, `UNIT_*` and `IDENTITY` are `inline constexpr`.
- Vector and matrix arithmetic, `cross`, `dot`, `translate`, `scale`, `transpose`, `determinant` and `inverse` work in constant expressions, as do quaternion arithmetic, `dot`, `conjugate` and `inverse`.
- Functions that call `std::sqrt` or the trig functions (`length`, `normalize`, rotations) can't run at compile time until C++26 makes those functions `constexpr`.

```
constexpr quat r { 0, 0, 0, 1 };                        // 180° about z
static_assert( r * vec3::UNIT_X == -vec3::UNIT_X );
static_assert( cross( vec3::UNIT_X, vec3::UNIT_Y ).z == 1.0f );
```

SSE dot product

`_mm_dp_ps` (SSE4) and `_mm_hadd_ps` (SSE3) are replaced by one SSE1 version: a multiply, two shuffle-adds and `_mm_cvtss_f32`. On MSVC, `Vector4f` `dot` went from 1.33 to 0.62 ns, the same as glm.

Behaviour changes to check

> These changes can break or change existing code that uses FastMath. Everything else is source-compatible.

CHANGE	EFFECT
`BREAKING` Vector aliases	`v.r/g/b/a`, `v.s/t`, `v.u/v` and `v.vec[i]` are gone. Use `x/y/z/w` or `v[i]`. Vectors with other sizes than 2, 3 or 4 store `elements[N]`.
`BREAKING` `Vector4f` and `__m128`	No more `.v` member, constructor from `__m128`, or conversion to `__m128`. Use `_mm_load_ps( &v.x )` and `_mm_store_ps( &v.x, r )`.
`BREAKING` Matrix members	`m.X/Y/Z/W` and the flat `m.m[i]` are gone. Use `m[0]` … `m[3]` for rows, `m[i][j]` for elements, or `m.data()[i]` for flat access.

CHANGE	EFFECT
BREAKING Macros	`LS_*` are now `FM_*`. Code that sets `LS_HANDEDNESS` or `LS_DEPTH_RANGE` before including FastMath must use the `FM_` names.
BREAKING CMake options	`FASTMATH_*` options are now `FM_*`. Existing build directories keep the old cache entries; reconfigure with the new names.
BREAKING Mixed vector sizes	`Vector4f + Vector3f` used to compile (the right side was silently converted). It's now an ambiguity error.
BREAKING Mixed precision	`quat * dquat` and `Vector3f * Vector<double, 3>` no longer compile implicitly. Convert with the converting constructor.
CHANGED Scalar conversion	`Vector3i * 2.5f` converts `2.5f` to `int` like any function call does (you get the normal conversion warning) instead of casting silently.
CHANGED Matrix `/ 0`	Asserts in Debug. In Release it divides (giving inf/NaN) instead of returning a zero matrix.
CHANGED Quaternion `-q`	Returns the negation, not the conjugate. Code that relied on the old behaviour should call `conjugate(q)`.
CHANGED `Transform`	The first `getMatrix()` after construction now reflects the constructor arguments (before, it was the identity).
CHANGED CMake target	Link `FastMath::FastMath`. AVX2 is enabled for linking targets only when FastMath is the top-level project; set `FM_ENABLE_AVX2=ON` when embedding it.

Verification

CONFIGURATION	RESULT
MSVC (VS 18), Release, AVX2	148 / 148 tests pass; benchmarks and example build
MSVC (VS 18), Debug, AVX2	148 / 148
MSVC (VS 18), Release, AVX2 off	148 / 148
GCC 11.4 (WSL Ubuntu), Debug	148 / 148, with `-Wall -Wextra -Werror -pedantic`

CONFIGURATION	RESULT
GCC 11.4 (WSL Ubuntu), Release	148 / 148
Clang	**Not tested.** It isn't installed locally, and the GitHub workflow only runs for pushes and pull requests to `main`. Opening a pull request will run the Clang jobs.

New tests in this round:

- `Vector.Constexpr_NamedComponents`: `static_assert` on `.x` through `.w`.
- `Vector.Data` and `Matrix.Rows`: the row layout and `data()`.
- `Quaternion.SameRotation`.

Decisions for you

1. **Quaternion layout.** Quaternions still use the union, in `w, x, y, z` order. Should they get the same plain-member treatment, and switch to `x, y, z, w` (the order glm, DirectXMath and GPU code use)? This would remove the last `#pragma` warning suppressions and the `quat[]` workaround in the quaternion operators.
2. **The `assert` in `operator[]`.** It costs about 14% compile time in optimized builds that keep asserts on (rare; CMake's Release builds define `NDEBUG`). Keep it, which is my recommendation, or drop it for speed?

Next round

- **Performance:**
  - Closed-form 3×3 and 4×4 `inverse` (about 10× slower than glm today).
  - `normalize` (2× slower than glm).
  - Matrix × vector on GCC (3× slower than glm).
  - NEON for ARM.
- **Missing game functions:** `min`, `max`, `clamp`, `lerp`, `reflect` and others.
- **Concepts** into the `FastMath` namespace.
- **A compile-time comparison** with glm and DirectXMath.
- **Interactive SDL3 samples.**

Files

FILE	CHANGE
`inc/FastMath/VectorBase.hpp`	Plain members and `operator[]`; no union or pragmas
`inc/FastMath/Vector.hpp`	Hidden friends, helper structs removed, constexpr constants, SSE dot, swizzle fix, operators build results directly
`inc/FastMath/MatrixBase.hpp`	`Vector<T, M> row[N]`; no union or pragmas
`inc/FastMath/Matrix.hpp`	Hidden friends, free `transpose/determinant/adjugate/inverse`, row-based algorithms, multiply and division fixes, `dmat2x4`, includes
`inc/FastMath/Quaternion.hpp`	Hidden friends, unary minus, `slerp`, `pitch`, constexpr arithmetic, `sameRotation`
`inc/FastMath/QuaternionBase.hpp`, `Config.hpp`	`FM_*` macros
`inc/FastMath/FastMath.natvis`	Matrices shown as rows
`inc/FastMath/Transform.hpp`	`isDirty` fix
`inc/FastMath/Common.hpp`	`deadzone`, `<cmath>`
`inc/FastMath/Concepts.hpp`	Doc comment namespace
`inc/FastMath/Math.hpp`	Deleted (duplicate)
`CMakeLists.txt`	INTERFACE target, `FM_*` options, warnings, natvis
`src/CMakeLists.txt`, `src/FastMath.cpp`	Deleted; the latter moved to `tests/Instantiation.cpp`
`tests/CMakeLists.txt`	Links the target, gtest CRT fix, warnings
`tests/*Tests.cpp`	25 new tests; `TransformTests.cpp` is new; matrix tests use rows
`perf/CMakeLists.txt`, `perf/VectorPerf.cpp`, `perf/MatrixPerf.cpp`	FetchContent glm, link the target, new matrix/quaternion/vector benchmarks

FILE	CHANGE
`examples/01-Vectors/CMakeLists.txt`	Links `FastMath::FastMath`
`.github/workflows/cmake-multi-platform.yml`	`-DFM_BUILD_TESTING=ON`
`reports/FastMath-Change-Report.pdf`	This report

Written by Claude (Claude Code) for review by the repository owner. All measurements were taken on the development machine on 29 October 2026.